

Accumulator-Based CPU Architecture

Key Components of an Accumulator-Based CPU

Accumulator (ACC)

- A special-purpose register that stores intermediate results during arithmetic and logic operations.
- Most operations are performed between the accumulator and other memory or registers.

Arithmetic Logic Unit (ALU)

- Performs arithmetic operations (e.g., ADD, SUB) and logical operations (e.g., AND, OR).
- The result is typically placed back into the accumulator.

Registers

- In addition to the accumulator, the CPU may have general-purpose or special-purpose registers.
- However, the accumulator is the central register used for computations.

Memory

- Stores data and program instructions.
- Instructions are fetched from memory, and data operands are often loaded into the accumulator.

Control Unit

- Coordinates the execution of instructions by generating control signals for ALU, memory, and registers.
 - Responsible for instruction fetch, decode, and control signal generation.
-

Working of an Accumulator-Based CPU

1. Loading Data

- Data is loaded from memory into the accumulator using instructions like **LOAD**.

2. Performing Operations

- Operations such as **ADD** and **SUB** use the value in the accumulator and a value from memory.

3. Storing Results

- Results after operations are stored back into the accumulator.
- The result can be stored into memory using instructions like **STORE**.

4. Example Instruction Sequence

- **LOAD A** : Load value at memory location A into ACC.
 - **ADD B** : Add value at memory location B to ACC.
 - **STORE C** : Store value of ACC into memory location C.
-

Pros and Cons of Accumulator-Based Architecture

Advantages

- **Simplicity:** Centralized operations reduce hardware complexity.
- **Compact Instruction Set:** Instructions involve fewer operands (mostly one).

Disadvantages

- **Limited Parallelism:** Since all operations depend on ACC, simultaneous execution is restricted.
 - **Performance Bottleneck:** Excessive use of a single register (ACC) may slow down performance.
-

Example Instruction Set (Hypothetical)

- **LOAD R** : Load value from memory/register R into ACC.
 - **ADD R** : Add value in R to ACC.
 - **SUB R** : Subtract value in R from ACC.
 - **STORE R** : Store ACC's value into memory/register R.
 - **JUMP addr** : Jump to instruction at address **addr**.
-

Accumulator-Based Architecture: 1-Operand Machine

Key Characteristics

- All operations involve the accumulator implicitly.
 - Instruction format: $I = \text{op.adr}$, where:
 - **op** = operation code
 - **adr** = memory address of the operand
 - Second operand is always the accumulator.
-

Instruction Cycle (Step-by-Step)

Notation

- **PC** = Program Counter
- **IR** = Instruction Register
- **AR** = Address Register
- **ACC** = Accumulator
- $M[]$ = Memory access notation

Step 1: Fetch

$IR \leftarrow M[PC]$ (Fetch instruction)
 $PC \leftarrow PC + 1$ (Increment PC)
 $op \leftarrow IR[opcode]$
 $AR \leftarrow IR[address]$

Step 2: Decode and Execute

LOAD adr: $ACC \leftarrow M[adr]$
ADD adr: $ACC \leftarrow ACC + M[adr]$
SUB adr: $ACC \leftarrow ACC - M[adr]$
STORE adr: $M[adr] \leftarrow ACC$
JUMP adr: $PC \leftarrow adr$

Example Execution

Initial Memory State

Address	Contents
100	LOAD 200
101	ADD 201
102	STORE 202
200	5
201	3
202	0

Execution Steps

- **PC = 100:** Fetch LOAD 200 $\Rightarrow ACC \leftarrow M[200] = 5$
 - **PC = 101:** Fetch ADD 201 $\Rightarrow ACC \leftarrow 5 + M[201] = 8$
 - **PC = 102:** Fetch STORE 202 $\Rightarrow M[202] \leftarrow 8$
-

Summary

- Operations revolve around a single accumulator.
- Simple and efficient for basic systems.
- Instruction format: `op.adr`, using one explicit operand.
- Suitable for early computers or embedded systems with limited resources.